Keep the grid stable

Dynamic topology optimization & converter tuning

TenSyGrid 1-day hackathon challenge proposal · Version 0.1

Universitat Politècnica de Catalunya · CETPartnership Horizon Europe (GA 101069750)

1 Background and motivation

Power systems across Europe are undergoing the most radical transition since the inception of alternating current networks. Massive integration of renewable energy sources is progressively displacing traditional synchronous generation in favor of **Inverter-Based Resources (IBRs)**, including utility-scale wind farms, solar PV arrays, and battery energy storage systems (BESS).

While power electronic converters allow high controllability, their fast dynamics and lack of physical inertia radically alter power system stability. Transmission System Operators (TSOs) must constantly navigate two conflicting operational priorities:

- **Static efficiency:** Minimizing ohmic active power transmission losses ($\sum P_{\text{loss}}$) and relieving line congestion by optimizing the network topology (line switching / breaker opening and closing).
- **Dynamic stability:** Guaranteeing that the resulting operating point does not trigger sub-synchronous resonances, control-loop interactions, or undamped electromechanical oscillations (min $\zeta \geq 5\%$).

The European research project **TenSyGrid** (*Tensors for System Analysis of Converter-Dominated Power Grids*) addresses this challenge through a mathematical paradigm shift: formulating grid dynamics into **hybrid implicit multilinear time-invariant (iMTI) models represented as multi-way tensors**. While classical linear state matrices fail to capture converter saturations and switching, and nonlinear EMT simulations are too slow for real-time operation, **low-rank tensor decompositions (such as Canonical Polyadic / CP decomposition)** break the curse of dimensionality. By compressing the state-parameter space into decomposed tensor formats, stability margins and modal properties can be evaluated orders of magnitude faster. Working natively with the open-source **VeraGrid** platform, this hackathon challenges teams to apply these principles to find optimal, stability-certified grid operating points.

2 The challenge overview

Mission statement. Participating teams are given a benchmark transmission grid in native `.veragrid` format with multiple candidate corridors and converter-interfaced generation. The hackathon starts with an ungraded one-hour warm-up reconstructing $\mathbf{f}(\mathbf{x}) = \Phi\,\mathbf{m}(\mathbf{x})$ from supplied matrices S and Φ, before moving to grid optimisation. Your mission is to:

1. **Reconstruct the function** encoded by S and Φ: return readable expressions and a callable numerical evaluator.
2. **Decide line breaker switching** ($u_{\text{line},k} \in \{0, 1\}$) to minimize steady-state transmission losses.
3. **Tune converter control parameters** (virtual damping, droop gains, voltage controls) to ensure the system is small-signal stable.
4. **Automate your strategy across 300 test scenarios** with variable renewable generation, loads, and $N - 1$ line outages.

Target audience. This hackathon is aimed at **Bachelor's, Master's and PhD students** in Electrical Engineering, Industrial Engineering, and Computer Science with an interest in power systems. Students in Physics or Mathematics are also welcome, although a background in these fields is recommended rather than required. Participants should be comfortable with Python; familiarity with power flow, dynamic modelling, or numerical linear algebra is helpful but not essential.

Team format and recommended profiles. Teams are formed by 2–4 participants. A multidisciplinary skill set is strongly encouraged: combining expertise in power systems and converter control (system dynamics, droop controls, modal analysis) with strong Python scripting and algorithmic optimization skills.

2.1 Phase-by-phase roadmap

Phase 1: Ungraded reconstruction warm-up (09:15–10:15)

Focus: From the structural representation to equations. Given $S \in \mathbb{R}^{N\times M}$ and $\Phi \in \mathbb{R}^{E\times M}$, reconstruct $\mathbf{f}(\mathbf{x}) = \Phi\,\mathbf{m}(\mathbf{x})$ using the factor convention and worked example in Section 2.2.

- **Output:** Readable component expressions and a callable Python evaluator.
- **Validation:** Compare numerical outputs with supplied reference cases, including constants, zero inputs and signed factors. Solving for roots is not required.
- **Milestone:** Review the reference results with mentors at 10:15. The warm-up is ungraded, requires no graded submission, and does not contribute to the final score.

2.2 Warm-up instructions and worked example

This one-hour warm-up is ungraded and does not contribute to the final score. Given a structural matrix S and a coefficient matrix Φ, **reconstruct the vector-valued function f that they encode**. Build a Python routine that returns readable expressions for its components $f_1, \ldots, f_E$ and a callable evaluator:

$$\boxed{\mathbf{f}(\mathbf{x}) = \Phi\,\mathbf{m}(\mathbf{x}).}$$

The inputs defining the function are S and Φ. The variables $\mathbf{x} = (x_1, \ldots, x_N)^\mathsf{T}$ follow the row order of S; numerical values of $\mathbf{x}$ are supplied only when evaluating the function.

Matrix convention. Let N be the number of variables, M the number of product terms, and E the number of equations. The input dimensions are

$$\mathbf{x} \in \mathbb{R}^N, \qquad S \in \mathbb{R}^{N\times M}, \qquad \Phi \in \mathbb{R}^{E\times M}.$$

Rows of S identify variables; columns identify terms. Column j of S and column j of Φ refer to the same term. Row k of Φ supplies its coefficients in equation k. Preserve the supplied variable order.

For this challenge, use the factor convention of the supplied multilinear tools:

$$a_{ij}(\mathbf{x}) = S_{ij}x_i + 1 - |S_{ij}|, \qquad m_j(\mathbf{x}) = \prod_{i=1}^{N} a_{ij}(\mathbf{x}), \qquad f_k(\mathbf{x}) = \sum_{j=1}^{M} \Phi_{kj} m_j(\mathbf{x}).$$

An entry $S_{ij} = 0$ gives a factor of 1; $S_{ij} = 1$ gives x_i; $S_{ij} = -1$ gives $-x_i$. Entries encode factors, **not powers**. With binary S, each column simply selects which variables appear in a monomial. An all-zero column represents the constant term 1. Each variable appears at most once in a term, so the representation is multi-affine in $\mathbf{x}$.

Reconstruction and example. For each column of S, build its product term. Multiply by the corresponding coefficients in each row of Φ, then sum to obtain each component f_k. Reconstructing **f** does not require solving $\mathbf{f}(\mathbf{x}) = \mathbf{0}$.

Worked example. For $\mathbf{x} = (x_1, x_2)^\mathsf{T}$, take

$$S = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix}, \qquad \Phi = \begin{pmatrix} -1 & 1 & 0 & 2 \\ 0 & 0 & 3 & -1 \end{pmatrix}.$$

Then $\mathbf{m} = (1, x_1, x_2, x_1x_2)^\mathsf{T}$, giving

$$\boxed{f_1(\mathbf{x}) = -1 + x_1 + 2x_1x_2, \qquad f_2(\mathbf{x}) = 3x_2 - x_1x_2.}$$

At $\mathbf{x} = (1, 2)^\mathsf{T}$, the expected output is $\mathbf{f}(1, 2) = (4, 4)^\mathsf{T}$.

Check your result. At 10:15, compare your expressions and evaluator with the reference results and ask mentors about any differences before moving to grid optimisation. There is no graded warm-up submission. Check constant terms, zero-valued inputs, signed factors and rectangular systems. Work with the M supplied columns rather than building all 2^N possible monomials.

Phase 2: Static topology and power dispatch (morning)

Focus: Steady-state optimization. Given candidate line corridors and fixed demand, teams decide which breakers to keep closed or opened and schedule generation setpoints.

- **Tool:** Solve the non-linear AC power flow via VeraGrid's Python API (`power_flow()`).
- **Constraints:** Bus voltage magnitudes must stay within $0.95 \leq V_i \leq 1.05$ p.u., and line flows must not exceed thermal ratings.
- **Target:** Minimize total active power transmission losses $P_{\text{loss}} = \sum_k P_{\text{loss},k}$.

Phase 3: Converter control tuning and dynamic stability (midday)

Focus: Small-signal stability. Opening lines alters the electrical distance and impedance between converter-interfaced resources. The dynamic layer is activated!

- **Tensor & Descriptor Model:** Converter interactions and parameter variations exhibit a non-linear structure that can be multilinearized. VeraGrid expresses the grid as a descriptor pencil $\mathbf{E}\,\dot{\mathbf{x}} = \mathbf{A}\,\mathbf{x}$, where **A** stores the linearized interactions between the parameter space **p** (droops, damping, virtual inertia).
- **Modal Analysis:** Compute generalised eigenvalues $\det(\mathbf{A} - \lambda\mathbf{E}) = 0$, where $\lambda_i = \sigma_i \pm j\omega_i$.
- **Actions:** Teams tune converter control parameters (frequency droop R_p, virtual damping D_v, and voltage gains K_v, K_p^{pll}).
- **Dynamic Target:** Ensure strict asymptotic stability ($\max \sigma_i < -10^{-5}$) and maximize the minimum damping ratio:

$$\zeta_i = \frac{-\sigma_i}{\sqrt{\sigma_i^2 + \omega_i^2}} \geq 5\% \quad \text{for all oscillatory modes } (f_i > 0.05\,\text{Hz}).$$

Phase 4: Automated multi-scenario batch sweep (afternoon)

Focus: Algorithmic resilience and speed. Real power systems never operate at a single fixed snapshot!

- **Stress Test:** The evaluation engine tests each team's script against a batch of $N = 300$ operating conditions: variable renewable penetration (from 20% to 100%), demand fluctuations, and $N - 1$ line contingencies.
- **Tensor-Accelerated Automation:** Leveraging low-rank tensor representations and VeraGrid batch capabilities, teams must deliver an automated Python script that swiftly evaluates multi-scenario stability and outputs certified low-loss configurations without manual intervention. The integrated TenSyGrid tensor engine — already available in the repository — provides the $(\mathbf{S}, \mathbf{\Phi})$ evaluators to avoid re-assembling system matrices for every scenario.

3 Evaluation metrics and scoreboard

At the end of the hackathon, an automated scoring harness runs each team's submission against a hidden evaluation test suite of 100 operating points.

3.1 Composite scoring function

The grid-optimisation score (Phases 2–4) is computed as:

$$\text{Score} = 0.35 \cdot \mathcal{S}_{\text{damping}} + 0.25 \cdot \mathcal{S}_{\text{loss}} + 0.25 \cdot \mathcal{S}_{\text{robustness}} + 0.15 \cdot \mathcal{S}_{\text{changes}} \tag{1}$$

where:

- $\mathcal{S}_{\text{damping}} = \min\left(1,\ \max\left(0,\ \frac{\min_i \zeta_i}{\zeta_{\text{ref}}}\right)\right)$ with $\zeta_{\text{ref}} = 10\%$: Minimum damping ratio across all electromechanical and converter modes, normalised to $[0, 1]$ (0 if any mode is unstable).
- $\mathcal{S}_{\text{loss}} = \max\left(0, 1 - \frac{P_{\text{loss}}}{P_{\text{loss, baseline}}}\right)$: Relative loss reduction achieved compared to the unoptimized baseline grid.
- $\mathcal{S}_{\text{robustness}} = \frac{N_{\text{stable}}}{N_{\text{total}}}$: Proportion of scenarios surviving without voltage violations or instability.
- $\mathcal{S}_{\text{changes}} = 1 - \frac{n}{n_{\max}}$: Linear reward for minimal structural changes relative to each scenario's initial grid. Here n is the number of actions performed and $n_{\max}$ is the maximum number of changes possible in that scenario, specified by the scenario definition and shared by all teams. Each line connection or disconnection and each model addition or removal counts as one action, with equal weight. Variables and equations added or removed as a consequence of that action are not counted separately. Replacing a model counts as two actions (one removal and one addition); adjusting existing control parameters or generation setpoints does not count. By construction, $0 \le n \le n_{\max}$. Zero actions gives full marks; reaching $n_{\max}$ gives zero. If $n_{\max} = 0$, no changes are possible and the criterion is defined as 1. It is computed per scenario and averaged over the hidden evaluation suite.

3.2 Scoring breakdown table

Criterion	Weight	Description
Dynamic stability	**35%**	Small-signal damping margin (ζ_{min}) and absence of undamped modes.
Static loss reduction	**25%**	Active power transmission loss minimization via topology switching.
Multi-scenario robustness	**25%**	Stability and constraint compliance across 100 hidden stress scenarios.
Minimal grid changes	**15%**	Linear reward $1 - n/n_{\text{max}}$ for fewer line switching and model addition or removal actions; n_{max} is set by each scenario.

Jury sovereignty. The evaluation jury consists of faculty and senior researchers from the participating TenSyGrid consortium institutions. While scores and rankings are primarily computed by the automated evaluation harness, the jury remains fully sovereign: strictly in exceptional or extreme circumstances (e.g., algorithmic exploits, numerical singularities, or uncaptured technical merit), the panel reserves the right to review and determine final standings.

4 1-day hackathon schedule

The hackathon is structured to run in parallel with the consortium's plenary sessions, with rotating mentor support from TenSyGrid researchers:

Time	Session and milestone
09:00 – 09:15	**Hackathon kick-off:** Challenge presentation by Eduardo Prieto Araujo, problem statement, stability criteria, and VeraGrid platform introduction.
09:15 – 10:15	**Ungraded warm-up: Reconstruct the function.** Build readable expressions and a Python evaluator for $\mathbf{f} = \Phi\,\mathbf{m}$ from S and Φ; validate on reference cases.
10:15 – 11:00	**Part 2: Static grid and topology switching.** Run the baseline and optimise topology and dispatch with the VeraGrid Python API.
11:00 – 11:30	*Coffee break.*
11:30 – 12:00	**Hackathon support:** Dedicated mentoring and technical Q&A with TenSyGrid consortium researchers.
12:00 – 13:00	**Part 3: Dynamic layer and converter control tuning.** Computing eigenvalues, modal analysis, and stabilizing critical oscillation modes (parallel to Plenary WP1–WP2 overview).
13:00 – 14:00	*Lunch break.*
14:00 – 15:30	**Part 4: Automation and batch resilience.** Multi-scenario batch sweep across 300 test operating points and parameter optimization (parallel to Plenary WP3–WP5 overview).
15:30 – 16:00	*Coffee break and code freeze.* Submission of final scripts to the automated scoring harness.
16:00 – 17:00	**Hackathon wrap-up and winners announcement:** Team presentations, live leaderboard reveal, and awards ceremony.

5 Resources provided to participants

Each team will receive a starter Git repository containing:

- **Warm-up matrix examples**: The integrated reconstruction instructions in Section 2.2, plus reference input/output cases for S and Φ.
- `benchmark_grid.veragrid`: Benchmark grid with synchronous generators and grid-following/grid-forming converters.

- `starter_script.py`: Ready-to-run template demonstrating model loading, AC power flow execution, descriptor state matrix assembly, and eigenvalue calculation in VeraGrid.
- `evaluate_quick.py`: Fast validation tool evaluating a reduced subset of 5–10 representative scenarios for sub-second iterative feedback during prototyping.
- `evaluate_local.py`: Local validation tool that computes loss and damping KPIs across sample scenarios before final submission.
- **Documentation and cheatsheets**: Quick-reference guide on converter parameters (PLL, virtual inertia, droop) and modal analysis interpretation.
- **Dedicated mentors**: Support from TenSyGrid researchers (UPC, eRoots, HAW Hamburg, IWES, UM) throughout the day.
- **TenSyGrid tensor engine**: The iMTI toolbox — originally developed in MATLAB and migrated into the VeraGrid TenSyGrid Python codebase — provides the $(\mathbf{S}, \mathbf{\Phi})$ multilinear representation and low-rank CP tensor evaluators described in Section 6, enabling fast stability sweeps across the parameter space without re-assembling system matrices from scratch.

6 Why this matters: the tensor paradigm

In a 100% renewable power grid, transmission line switching and converter control are tightly intertwined: an uncoordinated line opening that minimizes steady-state losses might inadvertently push a nearby grid-forming converter into an unstable oscillatory mode.

Traditional tools force operators to choose between overly optimistic linear approximations or computationally prohibitive EMT time-domain simulations. TenSyGrid's answer is a **mathematical paradigm shift**.

From matrices to tensors

A classical linearised power system is described by a state matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$. When converter control parameters $\mathbf{p} = (p_1, \ldots, p_d)$ vary, the system becomes a **matrix-valued function** $\mathbf{A}(\mathbf{p})$, and evaluating stability across the parameter space naively requires one eigendecomposition per operating point. Discretising each of the d parameters into m values gives m^d operating points in total, so m^d eigendecompositions would be needed to evaluate the entire grid — the **curse of dimensionality**, whose cost grows exponentially with d.

Consider now a **multi-affine** (multilinear) polynomial system in N variables, i.e. each variable appears with power at most one. The number of monomials is then 2^N, so expanding the full basis has complexity $\mathcal{O}(2^N)$ — exponential again.

TenSyGrid avoids materialising this basis by rewriting it as a **multi-way (N-order) tensor**

$$\mathcal{M} = \begin{pmatrix} 1 \\ x_1 \end{pmatrix} \otimes \cdots \otimes \begin{pmatrix} 1 \\ x_N \end{pmatrix}.$$

This is a **rank-1 tensor written in Canonical Polyadic (CP) form**: although it has 2^N entries, it is stored and evaluated implicitly from only N factors, so its memory footprint and the cost of a contracted product grow linearly, $\mathcal{O}(N)$, rather than exponentially. The number of monomials itself is unchanged; the gain comes from never forming the dense tensor.

Defining the structural matrix $\mathbf{S}$ (which variables compose each monomial) and the unfolded coefficient matrix $\mathbf{\Phi}$ (the associated coefficients), the original system can be expressed as

$$\mathbf{f}(\mathbf{x}) = \mathbf{\Phi}\, \mathbf{m}(\mathbf{x}) = \mathbf{0}, \qquad \mathbf{m}(\mathbf{x}) \text{ generated from } \mathbf{S}, \tag{2}$$

and, thanks to the linear behaviour of tensor algebra, quantities such as the Jacobian are obtained through multi-way contracted products, without ever assembling the full dense Jacobian.

Last but not least, this format exploits the sparsity of the power grid — in both $\mathbf{\Phi}$ and $\mathbf{S}$ — to further reduce storage and computational time.

Hybrid implicit multilinear time-invariant (iMTI) descriptor models

Converter-dominated grids are not purely ordinary differential equation systems: algebraic network constraints couple the differential dynamics, leading to **descriptor (DAE) pencils**

$$\mathbf{E}\,\dot{\mathbf{x}} = \mathbf{A}\,\mathbf{x}, \qquad \det(\mathbf{A} - \lambda\,\mathbf{E}) = 0.$$

VeraGrid assembles the system matrices from physical component models and exposes them directly through its Python API. TenSyGrid then lifts this pencil family into the iMTI tensor framework, with the goal of **sweeping topology switches and converter parameters** while avoiding a full model re-assembly for every scenario. This is an active research target: the current toolchain provides the multilinear $\mathbf{S}/\mathbf{\Phi}$ representation and sparse evaluators, while compact tensor integrators remain under development.

Why it matters for this hackathon

The 300-scenario batch sweep in Phase 4 is the practical motivation: teams that exploit the multilinear structure — or simply profile and batch the VeraGrid API efficiently — can evaluate stability margins substantially faster than a brute-force eigensolve per scenario, and fast evaluation is exactly what real TSOs need for online, security-constrained dispatch.

Note: this document is a first iteration. The challenge definition and the scoring function will be refined and improved over the coming days.